Практикум 2.5 Приближенное решение дифференциальных уравнений

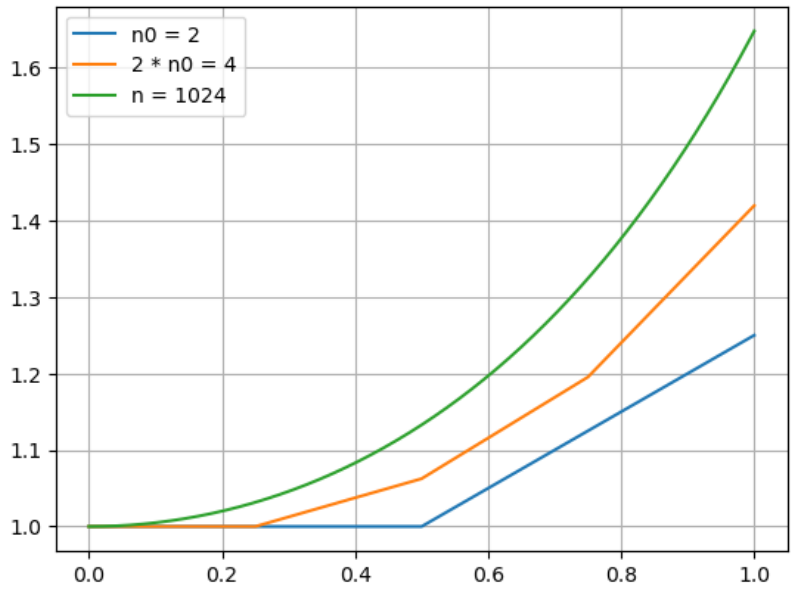
Упражнение 1.

Найти приближенное решение уравнения $y' = f(x, y)$, удовлетворяющего начальному условию $y(x_0) = y_0$, на отрезке $[a, b]$ ($a = x_0$) методом ломаных Эйлера с заданной точностью ε.

2. Для тестирования функции из п.1 используйте решение уравнения $y' = xy$ с начальным условием $y(0) = 1$ на отрезке $[0;1]$ с точностью $\varepsilon = 0{,}001$. Вначале найдите «вручную» точное решение.

```
def generate_dots(f, x0, y0, a, b, n):
    dx = (b - a) / n
    x_i = [x0 + dx * i for i in range(n + 1)]
    y_i = [y0]
    for i in range(1, len(x_i)):
        y_i.append(y_i[-1] + f.subs({x: x_i[i - 1], y: y_i[-1]}) * dx)

    return (x_i, y_i)

def dots_error(dots1, dots2):
    dy = []
    for i in range(len(dots1)):
        for k in range(len(dots2)):
            if (dots1[i][0] == dots2[k][0]):
                dy.append(abs(dots1[i][1] - dots2[k][1]))

    return max(dy)


def diff_euler(f, x0, y0, a, b, n0, eps):
    dots1 = generate_dots(f, x0, y0, a, b, n0)
    dots2 = generate_dots(f, x0, y0, a, b, 2 * n0)

    dots3 = dots1[::]
    dots4 = dots2[::]
    n = 2 * n0
    while (dots_error(dots3, dots4) >= eps):
        n = n * 2
        dots3 = dots4
        dots4 = generate_dots(f, x0, y0, a, b, n)

    plt.plot(dots1[0], dots1[1], label=f'n0 = {n0}')
    plt.plot(dots2[0], dots2[1], label=f'2 * n0 = {2 * n0}')
    plt.plot(dots4[0], dots4[1], label=f'n = {n}')
    plt.legend()

    return dots4, n
```

```
x = sp.Symbol('x')
y = sp.Symbol('y')

iter_solution, n = diff_euler(x * y, 0, 1, 0, 1, 2, 0.001)
plt.grid()
```



точное решение

$$\frac{dy}{dx} = xy$$

$$\frac{dy}{y} = xdx$$

$$\int \frac{dy}{y} = \int xdx$$

$$\ln|y| = \frac{x^2}{2} + \ln|C|$$

$$\ln|\frac{y}{C}| = \frac{x^2}{2} = \ln e^{\frac{x^2}{2}}$$

$$\frac{y}{C} = e^{\frac{x^2}{2}}$$

$$y = Ce^{\frac{x^2}{2}}$$

при y(0) = 1

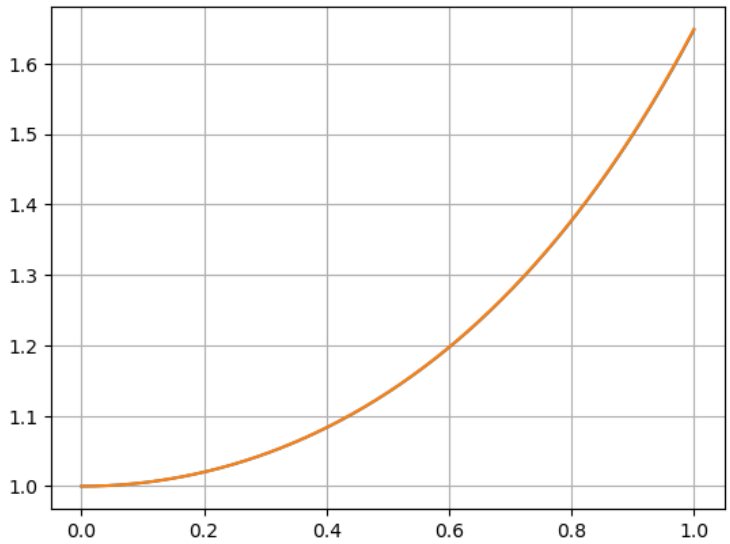
$$y = e^{\frac{x^2}{2}}$$

```
def sol(x):
    return np.exp(x**2 / 2)

difference = []
for i in range(len(iter_solution[0])):
    difference.append(abs(sol(iter_solution[0][i]) - iter_solution[1][i]))
```

```
print('оценка реальной точности приближения:', max(difference))
```

```
оценка реальной точности приближения: 0.00107251359680705
```

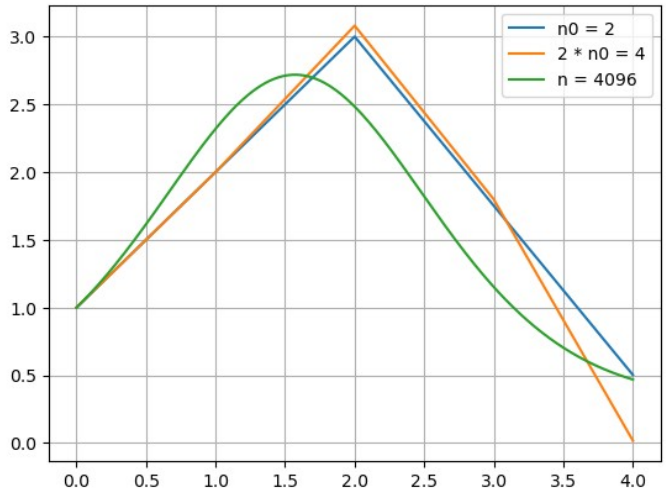
```
plt.plot(iter_solution[0], iter_solution[1])
x1 = np.linspace(0, 1, 1000)
plt.plot(x1, sol(x1))
plt.grid()
```



3. Найти приближенное решение уравнения $y' = f(x, y)$ из индивидуального задания, удовлетворяющего начальному условию $y(x_0) = y_0$, на отрезке $[a, b]$ ($a = x_0$) методом ломаных Эйлера с точностью $\varepsilon = 0{,}01$. Найти точное аналитическое решения.

Вариант 2: $y' = \cos(x)\, y$ при $y(0) = 1$ на отрезке $[0, 4]$ с точностью $\varepsilon = 0.001$.

```
x = sp.Symbol('x')
y = sp.Symbol('y')

iter_solution, n = diff_euler(sp.cos(x) * y, 0, 1, 0, 4, 2, 0.001)
plt.grid()
```



аналитическое решение

$$y' = \cos xy$$

$$\frac{dy}{dx} = \cos xy$$

$$\frac{dy}{y} = \cos x dx$$

$$\ln \frac{y}{C} = \sin x$$

$$y = Ce^{\sin x}$$

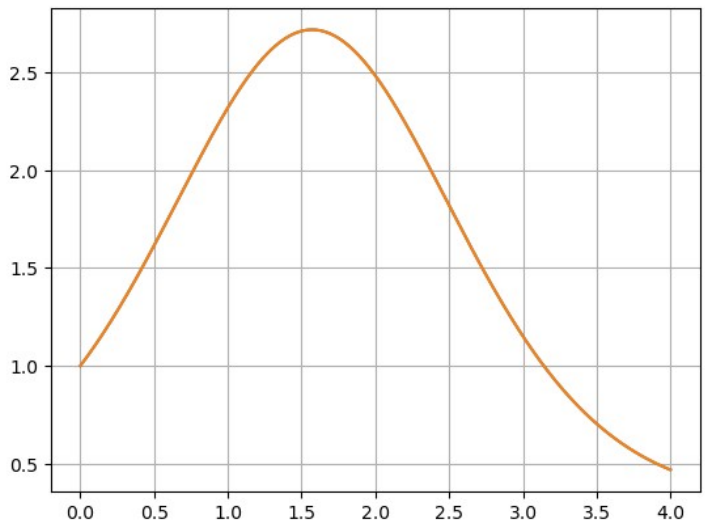
при $y(0) = 1$

$$y = e^{\sin x}$$

```
def sol(x):
    return np.exp(np.sin(x))

difference = []
for i in range(len(iter_solution[0])):
    difference.append(abs(sol(iter_solution[0][i]) - iter_solution[1][i]))

print('оценка реальной точности приближения:', max(difference))
```

```
оценка реальной точности приближения: 0.000796488852166544
```

```
plt.plot(iter_solution[0], iter_solution[1])
x1 = np.linspace(0, 4, 1000)
plt.plot(x1, sol(x1))
plt.grid()
```

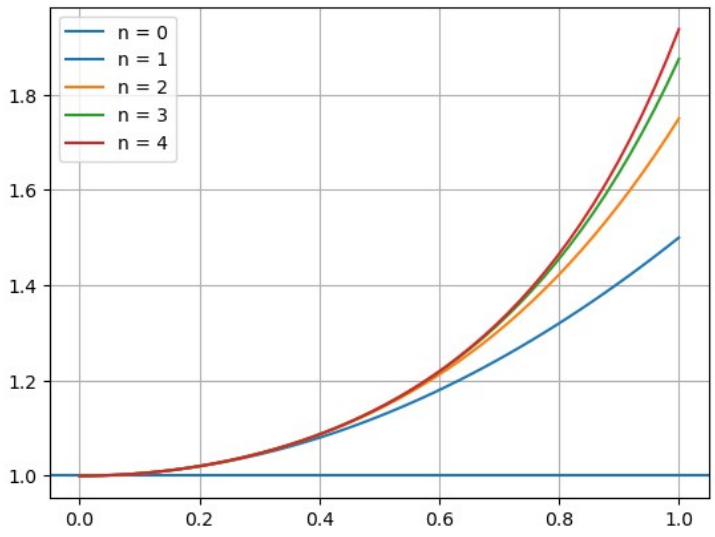


Найти приближенное решение уравнения $y' = f(x, y)$, удовлетворяющего начальному условию $y(x_0) = y_0$, методом последовательных приближений Пикара.

2. Для тестирования функции из п.1 используйте решение уравнения $y' = xy$ с начальным условием $y(0) = 1$ на отрезке $[0;1]$. Вначале найдите «вручную» точное решение.

```
def pikar(f, x, y, x0, y0, a, b, n0):
    t = sp.Symbol('t')
    phi = [y0]
    for i in range(1, n0):
        phi_i = y0 + sp.integrate(f.subs({x: t, y: phi[-1]}), (t, x0, x))
        phi.append(phi_i)

    x1 = np.linspace(a, b)
    plt.axhline(y0, label='n = 0')
    j = 1
    for i in phi[1::]:
        y1 = [i.subs(x, k) for k in x1]
        plt.plot(x1, y1, label=f'n = {j}')
        j += 1

    return phi
```

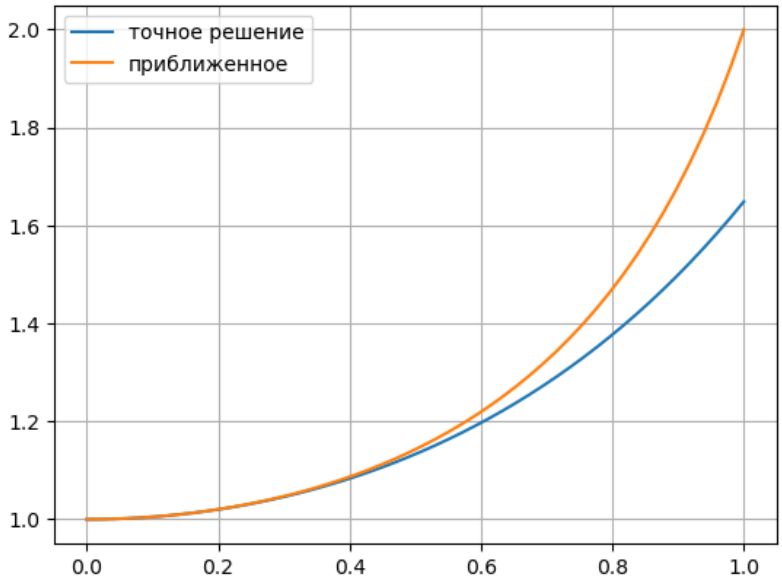
```
x = sp.Symbol('x')
y = sp.Symbol('y')

iter_s = pikar(x * y, x, y, 0, 1, 0, 1, 5)
plt.grid()
plt.legend()
```



```
def sol(x):
    return np.exp(x**2 / 2)

x1 = np.linspace(0, 1)
y1 = [iter_s[-1].subs(x, i) for i in x1]
y1_prec = sol(x1)
diff = [abs(i - k) for i, k in zip(y1, y1_prec)]

print('оценка реальной точности приближения для 5 итераций:', max(diff))
```

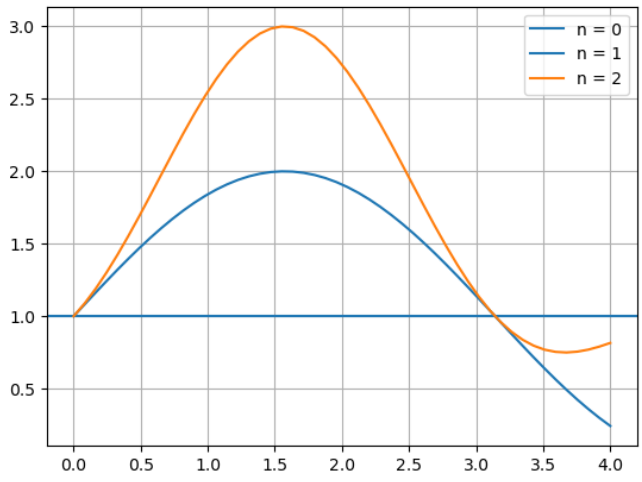
```
оценка реальной точности приближения для 5 итераций: 0.288778729299872
```

```
plt.plot(x1, y1, label='точное решение')
plt.plot(x1, y1_prec, label='приближенное')
plt.legend()
plt.grid()
```

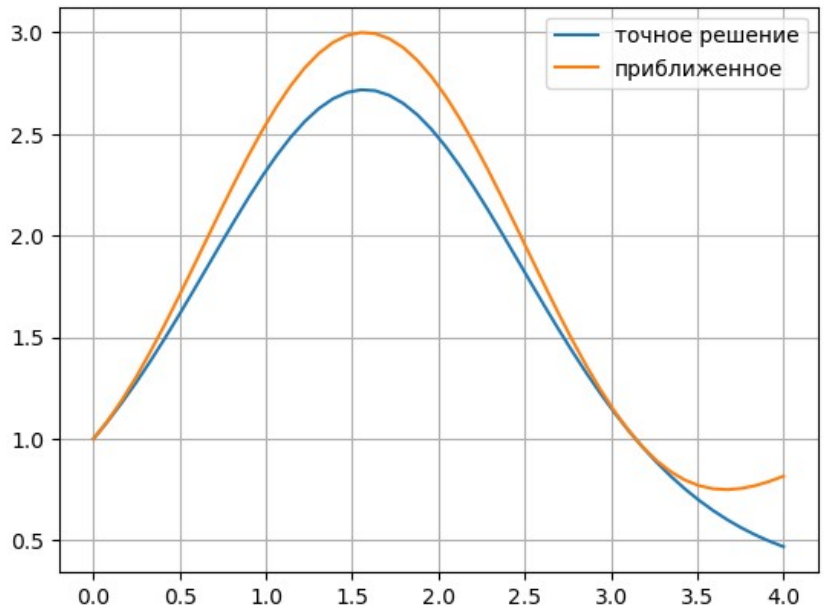


3. Найти приближенное решение уравнения из индивидуального задания методом последовательных приближений Пикара.

```
x = sp.Symbol('x')
y = sp.Symbol('y')

iter_s = pikar(sp.cos(x) * y, x, y, 0, 1, 0, 4, 3)
plt.grid()
plt.legend()
```




```
def sol(x):
    return np.exp(np.sin(x))

x1 = np.linspace(0, 4)
y1 = [iter_s[-1].subs(x, i) for i in x1]
y1_prec = sol(x1)
diff = [abs(i - k) for i, k in zip(y1, y1_prec)]

print('оценка реальной точности приближения для 3 итераций:', max(diff))
```

оценка реальной точности приближения для 3 итераций: 0.346783335722378

```
plt.plot(x1, y1_prec, label='точное решение')
plt.plot(x1, y1, label='приближенное')
plt.legend()
plt.grid()
```